CODING FOR DATA SCIENCE AND ANALYTICS

Revision Session: Loops, Functions, and Recursion

Exercise 1 — Sales Accumulator & Flowchart

Goal

Write a program that continuously asks the user to input the price of an item. The loop should stop when the user types the sentinel word **"totale"**.

Requirements

- **Step 1 (Design):** Before writing any Python code, draw a **Flowchart** (or use the Computational Thinking tripartition) to map out the algorithm's execution flow.
- **Step 2 (Code):** Translate your flowchart into code. Keep track of the total sum and the item count. When the loop ends, print the final receipt.

Exercise 2 — Tax Calculator

Goal

Define a function `calculate_net(gross, tax_rate)` that computes the net salary by subtracting the tax percentage from the gross amount.

Requirements

- Assign a **default value** of 20% (0.20) to the `tax_rate` parameter.
- Call the function **twice**:
  1. Provide only the gross salary (letting the function use the default tax rate).
  2. Provide both the gross salary and a custom tax rate (e.g., 35%).

Exercise 3 — Recursive Fibonacci

We previously calculated the Fibonacci sequence using loops. Now, let's use recursion.

Mathematical rules:

- **Base cases:** If $n = 0$, return 0. If $n = 1$, return 1.
- **Recursive case:** For $n > 1$, return $\text{fib}(n-1) + \text{fib}(n-2)$.

Task

Define the recursive function `fib_recursive(n)`. Call it to find the 10th Fibonacci number.
Discussion: What happens if you try to calculate the 50th number recursively? Why is it slower than the `for` loop?

Exercise 4 — The Collatz Conjecture

The Collatz conjecture is a famous unsolved mathematical problem.

The Rules

Start with any positive integer n.

- If n is even, divide it by 2.
- If n is odd, multiply it by 3 and add 1.

Repeat the process. The conjecture states you will **always** eventually reach 1.

Task (The Mega-Exercise)

1. Write a **recursive function** `collatz(n)` that prints the sequence and returns the number of steps it took to reach 1.
2. Write a main program with a **while loop and a sentinel**. Keep asking the user for a starting number until they type "esci", and print the sequence length for each number.